

Clase 098 — Cross-Site Request Forgery (CSRF)

Parte: 4 — Seguridad de aplicaciones web · Fuente: *The Web Application Hacker's Handbook (Stuttard & Pinto)* 🕒 Duración estimada: 90 min · Nivel: Intermedio

Objetivo

Entender el **CSRF (falsificación de petición entre sitios)**: forzar al navegador de una víctima autenticada a ejecutar acciones no deseadas. Aprenderás a construir PoCs, evaluar cuándo una defensa es efectiva y por qué SameSite y los tokens anti-CSRF funcionan.

⚠️ **Ética:** solo en labs propios/autorizados. Un PoC de CSRF ejecuta acciones reales sobre la cuenta de la víctima.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** las condiciones necesarias para un CSRF explotable.
2. **Construir** un PoC HTML (GET y POST) que dispare la acción.
3. **Evaluar** defensas: tokens anti-CSRF, SameSite, verificación de origen.
4. **Evadir** protecciones débiles (token no validado, método relajado).
5. **Recomendar** la defensa correcta según el contexto.

Temas

#	Tema	Por qué importa
1	Cómo funciona el CSRF	Base del ataque
2	Condiciones necesarias	Cuándo es explotable
3	PoC con formularios auto-enviados	Construcción del exploit
4	Tokens anti-CSRF	Defensa clásica
5	Cookies SameSite	Defensa moderna del navegador
6	Bypass de defensas débiles	Realidad de las apps
7	CSRF en APIs JSON	Matices con Content-Type

Definiciones y características

- **CSRF:** forzar una acción autenticada usando la sesión de la víctima desde otro sitio. Característica: explota que el navegador envía cookies automáticamente.

- **Token anti-CSRF**: valor impredecible ligado a la sesión que debe acompañar cada acción. Característica: el atacante no puede adivinarlo.
- **SameSite**: atributo de cookie que limita su envío entre sitios. Característica: `Lax` / `Strict` mitigan gran parte del CSRF.
- **Verificación de Origin/Referer**: comprobar el origen de la petición. Característica: defensa complementaria.
- **PoC (Proof of Concept)**: página que dispara la acción automáticamente. Característica: prueba el impacto real.
- **Double-submit cookie**: patrón donde el token va en cookie y en cuerpo. Característica: alternativa sin estado en servidor.

Herramientas y preparación

- **PortSwigger labs** de CSRF y **DVWA**.
- **Burp** (Community incluye un generador de PoC de CSRF).
- Un servidor local simple para alojar el PoC en tu laboratorio.

Laboratorio guiado

 Solo en labs propios.

1. En DVWA → *CSRF*, identifica la petición que cambia la contraseña.
2. Comprueba si requiere token anti-CSRF. Si no, es explotable.
3. Genera un PoC con Burp (clic derecho en la request → *Engagement tools* → *Generate CSRF PoC*).
4. Aloja el PoC en tu servidor de lab y ábrelo con una sesión de víctima activa:

```
<form action="http://dvwa.local/vulnerabilities/csrf/" method="POST">
  <input type="hidden" name="password_new" value="hacked">
  <input type="hidden" name="password_conf" value="hacked">
  <input type="hidden" name="Change" value="Change">
</form>
<script>document.forms[0].submit()</script>
```

5. Verifica que la contraseña cambió sin la interacción consciente de la víctima.
6. En un lab con token, intenta el **bypass**: quitar el token, reutilizar uno viejo, cambiar método.
7. Prueba el efecto de la cookie `SameSite=Lax` en el ataque.

Ejercicios

1. Construye un PoC GET y otro POST para la misma acción.
2. Explica por qué `SameSite=Strict` puede romper flujos legítimos.
3. Evade un token que se valida solo por presencia (no por valor).
4. Analiza si un endpoint JSON con `Content-Type: application/json` es CSRF-eable.
5. Diseña la defensa: token sincronizado + SameSite + verificación de Origin.
6. Diferencia CSRF de SSRF (nombres parecidos, ataques opuestos).

Reto verificable

Resuelve un lab de CSRF de PortSwigger que tenga una **defensa parcial** (token mal validado) y demuestra el cambio de email de la víctima. **Criterio de aceptación:** el lab queda resuelto, entregas el PoC funcional y explicas exactamente qué debilidad del token permitió el bypass y cómo se corrige.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El PoC no dispara la acción	Falta el token válido; el endpoint sí lo valida
SameSite bloquea el ataque	Cookie <code>Lax/Strict</code> ; el CSRF clásico ya no aplica
POST JSON no explotable	El navegador no envía JSON cross-site sin CORS
Token estático reutilizable	Debilidad real; repórtalo
Referer/Origin bloquean	La app valida origen; busca otro vector

Preguntas frecuentes

 **¿SameSite mató el CSRF?** Lo redujo mucho, pero no del todo: hay `SameSite=None` , flujos GET sensibles y navegadores/configuraciones variados. Mantén tokens.

 **¿Las APIs REST necesitan protección CSRF?** Si autentican por cookie, sí. Si usan tokens en cabecera (Bearer), el CSRF clásico no aplica.

 **¿Un token en URL sirve?** Es riesgoso: se filtra por Referer, logs e historial. Mejor en cuerpo o cabecera.

Referencias

- Stuttard & Pinto, *The Web Application Hacker's Handbook*, cap. 13.
- OWASP CSRF: <https://owasp.org/www-community/attacks/csrf>
- OWASP CSRF Prevention Cheat Sheet.
- PortSwigger CSRF: <https://portswigger.net/web-security/csrf>

Siguiendo clase

Clase 099 - Server-Side Request Forgery (SSRF)